



ASSIGNMENT MANAGER

Project Documentation · API Reference

Developer

Tanzid Mondol

Student ID: 0322320105101048

Course: CSE 3102 — Mobile Application Development

Instructor: Md. Forhan Shahriar Fahim

Pundra University of Science & Technology, Gokul, Bogura.

6th Semester — 2026

 **Instructor Note:** This document covers the complete project specification, architecture, Firestore data API, and all implementation decisions for CSE 3102 — Mobile Application Development coursework.

Table of Contents

1. Project Overview	...
2. Features	...
3. Tech Stack & Dependencies	...
4. Project Structure	...
5. Architecture	...
6. Data Model	...
7. API Reference — Firestore Operations	...
8. Notification Service API	...
9. Provider State API	...
10. Screens	...

11. Key Implementation Decisions	...
12. Firebase Setup Guide	...
13. Running the App	...
14. Future Improvements	...

1. Project Overview

Assignment Manager is a cross-platform mobile application built with Flutter and Firebase. It helps university students track, manage, and never miss assignment deadlines through smart notifications, real-time cloud sync, and a clean Material Design 3 interface.

Property	Details
Project Name	Assignment Manager
Developer	MD TANZID MONDOL
Student ID	0322320105101048
Course	CSE 3102 — Mobile Application Development
Instructor	Fahim Sir
University	Pundra University of Science & Technology
Semester	6th Semester — 2026
Platform	Flutter — Android / iOS / Web
Backend	Firebase (Firestore + Auth)
Language	Dart 3.x
Min Android SDK	21 (Android 5.0 Lollipop)
State Management	Provider (ChangeNotifier)
Version	1.0.0+1 (+1 = <i>first build version</i>)
Repository	https://github.com/tanzid-48/assignment_manager

 **Instructor Note:** The app solves a real student problem — missed deadlines. Every technical decision, from Provider state management to Firestore real-time sync, is motivated by this single goal.

2. Features

2.1 Core Assignment Management

- Create, read, update, and delete assignments (full CRUD)
- Fields: Title · Subject · Description · Priority · Status · Deadline
- Priority levels: High (red) · Medium (orange) · Low (green)
- Status: Pending (purple) · In Progress (blue) · Completed (green)
- Filter by: All · Pending · In Progress · Completed · High Priority
- Real-time search by title or subject — results update as you type
- Assignments sorted by deadline, earliest first

2.2 Smart Notification System

Trigger	Time	Notification
On assignment saved	Immediately	✅ Assignment Saved!
3 days before deadline	Deadline -3 days @ 09:00	 3 Days Left — start now!
1 day before deadline	Deadline -1 day @ 21:00	⚠️ Deadline Tomorrow!
On due day	Due day @ 08:00	🔴 Due Today — submit now!
On complete / delete	Immediately	(all notifications cancelled)

2.3 Firebase Backend

- Google Sign-In (OAuth 2.0) and Email/Password authentication

- Real-time Firestore sync — changes appear instantly across all devices
- Per-user data isolation enforced by Firestore security rules
- Cloud persistence — data survives app uninstall or phone replacement
- Password reset via Firebase email link

2.4 UI/UX Highlights

- Animated splash screen — "Made by Tanzid Mondol" with progress bar
- Bottom navigation: Home · Add · Stats · Settings
- Urgency banner on cards (due today / tomorrow / overdue in red)
- Statistics screen: status donut chart, subject bar chart, priority breakdown
- Dark mode with persistent local preference (shared_preferences)
- Smooth fade transitions between screens
- Form auto-resets and redirects to Home after saving

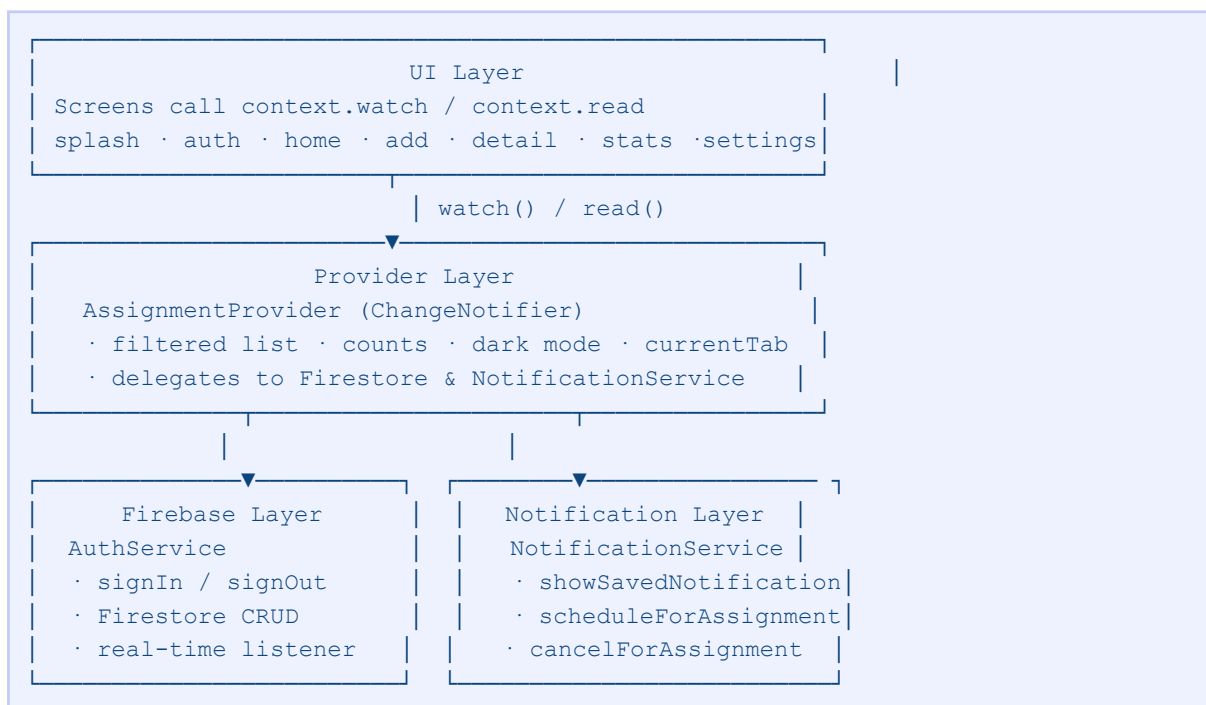
 **Instructor Note:** The UI is built entirely with Material Design 3 components — no third-party UI libraries. All color coding (priority, status, urgency) follows a consistent system defined in the provider layer.

3. Tech Stack & Dependencies

Package	Version	Purpose
flutter (SDK)	3.19+	Cross-platform UI framework
firebase_core	^3.6.0	Firebase SDK initialization
firebase_auth	^5.3.1	User authentication (Google + Email/Password)
cloud_firestore	^5.4.2	Real-time NoSQL cloud database
google_sign_in	^6.2.1	Google OAuth 2.0 sign-in flow
provider	^6.1.2	State management (ChangeNotifier pattern)
shared_preferences	^2.3.2	Local key-value store — dark mode preference

5. Architecture

The app follows a three-layer architecture with Provider-based reactive state management. Every screen uses `context.watch <AssignmentProvider>()` and rebuilds automatically when state changes — no manual `setState` required.



 **Instructor Note:** Provider was chosen over Bloc/Riverpod for simplicity and transparency. No manual `setState` is needed in most screens — `context.watch()` handles all reactive rebuilds automatically.

6. Data Model

6.1 Assignment Class (lib/models/assignment.dart)

Field	Dart Type	Firestore Type	Description
id	String	Document ID	Firestore auto-generated unique ID (not stored in document body)
title	String	string	Assignment title — required
subject	String	string	Course or subject name — required
description	String?	string null	Optional notes or details
deadline	DateTime	string (ISO 8601)	Due date — stored as ISO 8601 string
priority	String	string	"High" "Medium" "Low"
status	String	string	"Pending" "In Progress" "Completed"
createdAt	DateTime	string (ISO 8601)	Creation timestamp

6.2 Firestore Document Example

Collection path: `users/{uid}/assignments/{docId}`

```
{
  "title":    "Database ER Diagram",
  "subject":  "CSE 3102",
  "description": "Normalize to 3NF — include all relationships",
  "deadline": "2026-05-20T00:00:00.000",
  "priority": "High",
  "status":   "Pending",
  "createdAt": "2026-05-04T10:30:00.000"
}

// Note: "id" is the Firestore document key — NOT a field inside the document.
```

6.3 Firestore Security Rules

```
rules_version = "2";
service cloud.firestore {
  match /databases/{database}/documents {
    // Each user can only read/write their own assignments
    match /users/{userId}/assignments/{assignmentId} {
      allow read, write: if request.auth != null
                           && request.auth.uid == userId;
    }
  }
}
```

 **Instructor Note:** Security rules enforce that a logged-in user can only access documents under their own UID path. Unauthenticated requests are rejected at the Firestore level — not just inside the Flutter app.

7. API Reference — Firestore Operations

All communication with Firebase Firestore is documented below using REST-style conventions. Requests are authenticated automatically via the Firebase SDK using the current user's JWT token.

Base Path: `users/{uid}/assignments`

Auth: Firebase Auth JWT — auto-managed by SDK

Format: JSON — Firestore documents

Real-time: `Firestore.snapshots()` listener active after login

7.1 Get All Assignments

GET	<code>/users/{uid}/assignments?orderBy=deadline</code>
------------	--

Retrieves all assignments for the authenticated user, ordered by deadline ascending. Called on login (await) and maintained as a real-time. `snapshots()` listener for live updates.

Parameters

Name	Type	Required	Description
uid	string	Yes	Firebase Auth UID of the current user
orderBy	string	No	Sort field — defaults to "deadline" ascending

Dart Implementation

```
Future<void> load() async {
  final uid = FirebaseAuth.instance.currentUser!.uid;
  final snap = await FirebaseFirestore.instance
    .collection("users/$uid/assignments")
    .orderBy("deadline")
    .get();
  _assignments = snap.docs.map((d) {
    final data = d.data()..["id"] = d.id;
    return Assignment.fromMap(data);
  }).toList();
  notifyListeners();
}
```

💡 This operation runs twice: once with `await` for the initial load, and once as a real-time `.snapshots()` listener so the UI updates instantly when data changes on another device.

Responses

200 Returns array of Assignment documents ordered by deadline

401 Unauthenticated — no valid Firebase Auth token

403 Forbidden — uid in path does not match auth token

7.2 Get Single Assignment

GET`/users/{uid}/assignments/{assignmentId}`

Retrieves a single assignment document by its Firestore document ID. Used by DetailScreen.

Parameters

Name	Type	Required	Description
uid	string	Yes	Firebase Auth UID
assignmentId	string	Yes	Firestore document ID (auto-generated string)

Responses

200

Returns single Assignment document

404

Document not found or deleted

401

Unauthenticated

7.3 Create Assignment

POST`/users/{uid}/assignments`

Creates a new assignment document. Firestore auto-generates the document ID. After creation, schedules 3 local deadline notifications and shows an immediate "Saved" confirmation.

Parameters

Name	Type	Required	Description
title	string	Yes	Assignment title
subject	string	Yes	Course or subject name
description	string	No	Optional notes
deadline	ISO 8601	Yes	Due date — e.g. "2026-05-20T00:00:00.000"
priority	string	Yes	"High" "Medium" "Low"
status	string	Yes	"Pending" "In Progress" "Completed"
createdAt	ISO 8601	Yes	Creation timestamp — set to DateTime.now() on client

Dart Implementation

```
Future<void> add(Assignment a) async {
  final col = FirebaseFirestore.instance
    .collection("users/$uid/assignments");
  // POST — Firestore creates the document and returns a DocumentReference
  final doc = await col.add(a.toMap());
  final saved = a.copyWith(id: doc.id);
  _assignments.add(saved);
  await NotificationService.instance.scheduleForAssignment(saved);
  await NotificationService.instance.showSavedNotification(
    title: a.title, subject: a.subject,
  );
  notifyListeners();
}
```

💡 The returned `DocumentReference.id` becomes `Assignment.id` in local state. The `id` field is NOT stored inside the Firestore document body — it is the document key itself.

Responses

200

Assignment created — returns DocumentReference with auto-generated ID

400

Missing required fields (title or subject)

401

Unauthenticated

7.4 Update Assignment

PUT`/users/{uid}/assignments/{assignmentId}`

Updates an existing assignment document (full overwrite). Old notifications are cancelled and rescheduled for the updated deadline.

Parameters

Name	Type	Required	Description
assignmentId	string	Yes	Firestore document ID to update
title	string	Yes	Updated title
subject	string	Yes	Updated subject
description	string	No	Updated notes (null to clear)
deadline	ISO 8601	Yes	Updated due date
priority	string	Yes	Updated priority level
status	string	Yes	Updated status
createdAt	ISO 8601	Yes	Original creation timestamp — unchanged

Dart Implementation

```
Future<void> update(Assignment a) async {
  // PUT - overwrite all fields in existing document
  await FirebaseFirestore.instance
    .collection("users/${uid}/assignments")
    .doc(a.id)
    .update(a.toMap());
  final i = _assignments.indexWhere((e) => e.id == a.id);
  if (i != -1) _assignments[i] = a;
  await NotificationService.instance.scheduleForAssignment(a);
  notifyListeners();
}
```

💡 If status is set to "Completed", `NotificationService.cancelForAssignment(id)` is called to suppress all pending alerts.

Responses

200 Assignment updated — notifications rescheduled for new deadline

404 Document not found

401 Unauthenticated

7.5 Delete Assignment

DELETE `/users/{uid}/assignments/{assignmentId}`

Permanently deletes an assignment document from Firestore and cancels all 3 associated scheduled notifications.

Parameters

Name	Type	Required	Description
assignmentId	string	Yes	Firestore document ID to delete

Dart Implementation

```
Future<void> delete(String id) async {  
  // DELETE - remove Firestore document  
  await FirebaseFirestore.instance  
    .collection("users/$uid/assignments")  
    .doc(id)  
    .delete();  
  _assignments.removeWhere((e) => e.id == id);  
  await NotificationService.instance.cancelForAssignment(id);  
  notifyListeners();  
}
```

Responses

200	Assignment deleted — Firestore document removed, local state cleared, notifications cancelled
------------	---

404	Document not found
------------	--------------------

401	Unauthenticated
------------	-----------------

7.6 Auth — Google Sign-In

POST	/auth/google
-------------	--------------

Initiates Google OAuth 2.0 sign-in. On success, creates or retrieves a Firebase Auth user, starts the Firestore real-time listener, and loads assignments.

Dart Implementation

```
Future<UserCredential?> signInWithGoogle() async {
  final googleUser = await GoogleSignIn().signIn();
  if (googleUser == null) return null; // user cancelled
  final googleAuth = await googleUser.authentication;
  final credential = GoogleAuthProvider.credential(
    accessToken: googleAuth.accessToken,
    idToken: googleAuth.idToken,
  );
  return await FirebaseAuth.instance.signInWithCredential(credential);
}
```

💡 Google Sign-In requires the SHA-1 fingerprint of the signing certificate to be registered in Firebase Console for Android release builds.

Responses

200 UserCredential returned — user logged in, real-time listener started

400 User cancelled the Google sign-in picker

401 Google auth failed — invalid credentials

7.7 Auth — Email / Password

POST /auth/email

Signs in or registers a user with email and password. Returns friendly error messages for common Firebase Auth error codes.

Parameters

Name	Type	Required	Description
email	string	Yes	User email address
password	string	Yes	Password — minimum 6 characters

action	string	Yes	"login" "register"
--------	--------	-----	----------------------

Responses

200	UserCredential returned — authenticated successfully
400	weak-password / invalid-email / email-already-in-use
401	wrong-password
404	user-not-found — login with unregistered email

7.8 Auth — Password Reset

POST	/auth/reset
------	-------------

Sends a password reset email to the provided address via Firebase Auth.

Parameters

Name	Type	Required	Description
email	string	Yes	Registered email address

Responses

200	Reset email sent successfully
404	No account found for this email

400

Invalid email format

7.9 Auth — Sign Out

POST`/auth/signout`

Signs out from both Google Sign-In and Firebase Auth. Calls `AssignmentProvider.clearOnLogout()` to wipe local state. App returns to AuthScreen via stream.

Responses

200

Signed out — app navigates back to AuthScreen

 **Instructor Note:** All Firestore operations use the Firebase SDK — there is no custom HTTP server. "REST-style" documentation is used here for clarity and to demonstrate understanding of CRUD semantics required for CSE 3102.

8. Notification Service API

All notification logic lives in `lib/services/notification_service.dart`. Uses `flutter_local_notifications` with timezone-aware scheduling via the `timezone` package.

8.1 Methods

Method	Trigger	Description
<code>init()</code>	App startup in <code>main()</code>	Initialises the plugin, configures Android and iOS settings, requests notification permission on Android 13+.

showSavedNotification(t, s)	After add()	Shows an instant notification confirming the assignment was saved successfully.
scheduleForAssignment(a)	After add() and update()	Schedules up to 3 timed reminders (3 days before, 1 day before, on due day). Silently skips past-time slots.
cancelForAssignment(id)	After delete()/update()	Cancels all 3 scheduled notifications for the given assignment using its hashCode as the int ID base.
cancelAll()	Settings / manual reset	Cancels every scheduled notification across all assignments.

8.2 Notification Schedule

Notification	Scheduled Time	ID Formula
 3 Days Left!	deadline – 3 days @ 09:00	id.hashCode * 10 + 2
 Deadline Tomorrow!	deadline – 1 day @ 21:00	id.hashCode * 10 + 0
 Due Today!	Due day @ 08:00	id.hashCode * 10 + 1

8.3 Implementation

```

Future<void> scheduleForAssignment(Assignment assignment) async {
  await init();
  await cancelForAssignment(assignment.id); // clear previous reminders
  final now = DateTime.now();
  final d = assignment.deadline;

  // ① 3 days before - 09:00
  final t3 = DateTime(d.year, d.month, d.day, 9)
    .subtract(const Duration(days: 3));
  if (t3.isAfter(now)) await _schedule(
    id: _notifId(assignment.id, 2),
    title: "📅 3 Days Left!", body: "...", time: t3);

  // ② 1 day before - 21:00
  final t1 = DateTime(d.year, d.month, d.day, 21)
    .subtract(const Duration(days: 1));
  if (t1.isAfter(now)) await _schedule(
    id: _notifId(assignment.id, 0),
    title: "⚠️ Deadline Tomorrow!", body: "...", time: t1);

  // ③ Due day - 08:00
  final t0 = DateTime(d.year, d.month, d.day, 8);
  if (t0.isAfter(now)) await _schedule(
    id: _notifId(assignment.id, 1),
    title: "🔴 Due Today!", body: "...", time: t0);
}

// String → int ID helper (flutter_local_notifications requires int)
int _notifId(String id, int suffix) => id.hashCode * 10 + suffix;

```

 **Instructor Note:** flutter_local_notifications requires integer notification IDs. Since Firestore generates string document IDs, the _notifId() helper converts them to stable integers using String.hashCode — ensuring each assignment always maps to the same three notification IDs.

9. Provider State API

AssignmentProvider exposes these getters and methods. All screens use context.watch<AssignmentProvider>() to rebuild reactively.

9.1 Getters

Getter	Type	Description
assignments	List<Assignment>	Full unfiltered assignment list for the current user.

filtered	List<Assignment>	Search + filter + deadline-sorted list. Used directly by HomeScreen.
totalCount	int	Total number of assignments.
pendingCount	int	Assignments with status "Pending".
inProgressCount	int	Assignments with status "In Progress".
completedCount	int	Assignments with status "Completed".
isDarkMode	bool	Dark mode state — persisted in SharedPreferences.
currentTab	int	Active bottom navigation index (0=Home 1=Add 2=Stats 3=Settings).
filterStatus	String	Active filter chip value. Default: "All".
searchQuery	String	Active search string. Default: empty string.

9.2 Methods

Method	Returns	Description
load()	Future<void>	Fetches assignments from Firestore on login and starts real-time listener.
add(Assignment)	Future<void>	Saves to Firestore, schedules notifications, shows saved confirmation.
update(Assignment)	Future<void>	Updates Firestore document, reschedules notifications.
delete(String id)	Future<void>	Deletes from Firestore, cancels notifications, removes from local list.
setSearch(String q)	void	Updates search query — triggers immediate UI rebuild.
setFilter(String f)	void	Updates filter chip selection — triggers immediate UI rebuild.
setCurrentTab(int i)	void	Switches bottom navigation tab programmatically (used after Save).

toggleDarkMode()	Future<void>	Flips dark mode and persists to SharedPreferences.
clearOnLogout()	void	Resets all state to empty — called on sign out.

10. Screens

Screen	File	Description
Splash	splash_screen.dart	Animated launch screen — 3.2s fade to AuthWrapper
Auth	auth_screen.dart	Login and Register tabs. Google Sign-In + Email/Password. Forgot password flow.
Home	home_screen.dart	Stats header · search · filter chips · urgency banners · bottom nav hub
Add / Edit	add_assignment_screen.dart	Form for create or edit. fromNav:true switches tab; false pops Navigator.
Detail	detail_screen.dart	Full assignment view. Quick status update (3 buttons). Edit and Delete in AppBar.
Statistics	statistics_screen.dart	Status donut chart · subject bar chart · priority breakdown · upcoming list
Settings	settings_screen.dart	User info card · dark mode switch · notification reference · Sign Out

11. Key Implementation Decisions

Decision	Rationale
----------	-----------

Assignment.id is String (not int)	Firestore auto-generates string document IDs. Using int would require a manual counter — error-prone in a distributed system.
context.watch() in build() for tab	didChangeDependencies() runs only once per lifecycle, missing subsequent provider updates. watch() rebuilds reactively on every notifyListeners() call.
p.setCurrentTab(i) not setState()	Tab state lives in AssignmentProvider so the Add screen can trigger navigation to Home without needing a parent BuildContext.
SharedPreferences only for dark mode	Dark mode is a device-level preference. All assignment data belongs to the user account and must be cloud-synced via Firestore.
_notifId(String, int) for notification IDs	flutter_local_notifications requires integer IDs. String.hashCode.abs() converts a Firestore document ID to a stable, collision-resistant integer.
isAfter(now) guard before scheduling	Past-time notifications are silently skipped. A deadline already missed should not trigger new alerts.
Real-time .snapshots() listener	Keeps UI in sync if the user edits on another device or browser tab simultaneously without requiring a manual refresh.
Default deadline = DateTime.now()	Replacing the previous +7 day default with today forces intentional date selection, reducing accidental far-future deadlines.

 **Instructor Note:** Every decision listed above was discovered through a real bug or code review during development — not planned upfront. Each entry represents a committed fix in the GitHub repository.

12. Firebase Setup Guide

Follow these steps to configure Firebase for a fresh clone of this project.

Step 1 — Create Firebase Project

1. Go to console.firebase.google.com and sign in with your Google account
2. Click "Create a project" → name it "assignment-manager"
3. Disable Google Analytics → click "Create project"

Step 2 — Add Android App

4. Firebase Console →  Project Settings → "Your apps" → Android icon
5. Package name: `com.example.assignment_manager`
6. Download `google-services.json` → place at `android/app/google-services.json`

Step 3 — Update Gradle Files

`android/settings.gradle.kts` — add inside `plugins { }` block:

```
id("com.google.gms.google-services") version "4.4.2" apply false
```

`android/app/build.gradle.kts` — add inside `plugins { }` block:

```
id("com.google.gms.google-services")
```

`android/app/build.gradle.kts` — update `defaultConfig`:

```
minSdk = 21    // required - flutter.minSdkVersion is too low for Firebase
```

Step 4 — Install & Configure

```
# Install Flutter packages
flutter pub get

# Install FlutterFire CLI
dart pub global activate flutterfire_cli
npm install -g firebase-tools
firebase login

# Auto-generate firebase_options.dart
# (if "flutterfire" is not recognized, use the full command below)
dart pub global run flutterfire_cli:flutterfire configure
```

```
# → Select your project → select android → done
```

Step 5 — Enable Firebase Services

- Authentication → Sign-in method → Enable Google  and Email/Password 
- Firestore Database → Create database → Production mode → Region: asia-south1
- Apply the security rules from Section 6.3

 **Instructor Note:** google-services.json is git-ignored and must be added manually after cloning. This is intentional — the file contains project-specific API keys that must never be committed to a public repository.

13. Running the App

```
# 1. Clone the repository
git clone https://github.com/tanzid-48/assignment_manager.git
cd assignment_manager

# 2. Install dependencies
flutter pub get

# 3. Place google-services.json in android/app/ (see Section 12)

# 4. Run on connected Android device
flutter run -d android

# 5. Run on Chrome (Web)
flutter run -d chrome

# 6. Build release APK
flutter build apk --release
# Output: build/app/outputs/flutter-apk/app-release.apk

# 7. Build App Bundle (Google Play Store)
flutter build appbundle --release
```


14. Future Improvements

Frontend

- Swipe-to-delete and swipe-to-complete (Dismissible widget)
- Calendar view — monthly grid showing deadlines
- Subtasks / checklist inside each assignment
- File attachment — photo of assignment sheet
- Home screen widget for glanceable deadline summary
- Custom reminder time per assignment (user-configurable)
- Recurring assignments for weekly labs

Backend

- Server-side push via Firebase Cloud Messaging (FCM) — works when app is fully closed
- Offline support using Firestore local persistence cache
- Group / classmate assignment sharing
- Export to PDF / CSV for submission history
- Supabase migration option — open-source Firebase alternative

 **Instructor Note:** FCM push notifications would allow the server to deliver alerts even when the app is fully closed — replacing the current device-local scheduling with true server-side delivery.

Assignment Manager

MD:TANZID MONDOL

email: mdtanzid.525@gmail.com

Repo Link: https://github.com/tanzid-48/assignment_manager